Description Submission view

Available from: Cuma, 27 Şubat 2026, 11:06 AM
Due date: Cuma, 27 Şubat 2026, 12:36 PM
Requested files: towerDefense.java (Download)
Type of work: Individual work
Reduction by automatic evaluation: 25 **Free evaluations**: 3**Reduction by automatic evaluation**: 25 **Free evaluations**: 3

Tower Defense Simulation

Please read the class descriptions, edge cases, and scenarios carefully to complete your lab session successfully in the Moodle VPL environment.

Tower Class

The base class of our simulation. It represents a standard defensive structure placed on a 2D Cartesian coordinate system.
Attributes:

- x (double): The X-coordinate of the tower on the map.
- y (double): The Y-coordinate of the tower on the map.
- range (double): The maximum distance the tower can fire at an enemy.
- baseDamage (int): The standard damage the tower inflicts before any modifiers.

Methods:

- **Tower(double x, double y, double range, int baseDamage)**
  - *Description:* The constructor initializes a new tower with the given coordinates, range, and base damage.
- **calculateDistance(double enemyX, double enemyY)** returning double
  - *Description:* Calculates the Euclidean distance between the tower and the enemy's coordinates. It utilizes the standard mathematical distance formula: $\sqrt{(enemyX - x)^2 + (enemyY - y)^2}$
  - *Crucial Detail:* This calculation must be robust enough to handle negative coordinates flawlessly. Additionally, if an enemy is positioned exactly at the tower's coordinates, the distance should evaluate to 0.0.
- **canHit(double enemyX, double enemyY)** returning boolean
  - *Description:* Evaluates if an enemy is within the tower's firing range.
  - *Crucial Detail:* This check is inclusive. An enemy located *exactly* on the border of the range limit is considered a valid target and can be hit.

?

getBaseDamage() returning int

- *Description:* Returns the base damage of the tower.

SniperTower Class

This class inherits from the Tower class. It represents a specialized, high-damage tower with limited ammunition and critical strike capabilities.
Additional Attributes:

- ammo (int): The current number of bullets the tower holds.
- criticalMultiplier (int): The multiplier applied to the base damage during a critical hit.
- shotsFired (int): A private counter tracking how many shots the tower has fired (starts at 0).

Methods:

- **SniperTower(double x, double y, double range, int baseDamage, int ammo, int criticalMultiplier)**
  - *Description:* Initializes the base attributes by calling the parent constructor, and sets up the sniper-specific attributes.
- **fire(double enemyX, double enemyY, int enemyArmor)** returning int
  - *Description:* Handles the firing mechanism.
  - *Ammo Conservation:* If the enemy is out of range, or if the tower has 0 ammo, the method returns 0 damage **without** consuming any ammunition.
  - *Combat & Armor Logic:* If successful, it consumes exactly one ammo and increments the shotsFired. The enemy's armor is subtracted from the total damage. If the armor is higher than the incoming damage, the resulting damage defaults to 0 (preventing negative damage). Most importantly, applying armor reduction must *never* permanently alter the tower's base damage attribute.
  - *Critical Strike Mechanic:* Every 3rd shot fired (e.g., 3rd, 6th, 9th) triggers a critical hit, multiplying the base damage by the criticalMultiplier *before* armor is deducted. This tracking is independent of reloading.
- **reload(int bulletsToAdd)** returning void
  - *Description:* Restocks the tower's ammunition incrementally using a loop until the requested number of bullets is added. Passing a value of 0 is a valid operation and will simply result in no bullets being added.
- **engageEnemies(double[][] enemies)** returning int
  - *Description:* The core combat cycle. It accepts a 2D array of enemies [x, y, armor, health].
  - *Combat Logic:* The method iterates through all enemies. Out-of-range enemies are skipped entirely without wasting ammo. If in range, the tower fires repeatedly until the enemy is defeated.
  - *Auto-reload Check:* If ammunition depletes during an active engagement, the tower automatically calls the reload method for 5 bullets and continues firing.
  -

Defeat Conditions: An enemy is considered defeated if its health drops to exactly 0, or drops below 0 ("overkill").

- *Infinite Loop Protection:* If the tower's shots deal 0 damage (e.g., due to impenetrable armor), the tower ceases fire immediately, breaks the loop for that specific enemy to prevent an infinite cycle, and conserves its remaining ammunition.

Edge Cases & Simulation Scenarios

When writing your code, pay close attention to the VPL evaluation tests. Your logic will be tested against standard waves as well as specific edge cases.

Scenario 1: The Empty Wave

- **Concept:** The tower is engaged, but the provided 2D array of enemies is completely empty.
- **Expected Behavior:** The tower should elegantly handle the empty array, doing nothing, and simply return 0 defeated enemies.

Scenario 2: The Heavily Armored Enemy (Infinite Loop Prevention)

- **Concept:** A single enemy enters the range but possesses an armor rating much higher than the tower's maximum critical damage.
- **Expected Behavior:** The tower will fire its first shot, calculate 0 damage due to the armor, and the safety mechanism will trigger. It will stop attacking that invincible target, saving the rest of its ammunition, and move on. The enemy survives.

Scenario 3: Swarm Attack & Array Integrity

- **Concept:** A massive array of enemies is sent. Some are completely out of bounds (way beyond range), while others are clustered close by.
- **Expected Behavior:** The tower will systematically check each enemy. It must correctly skip the invalid/out-of-range elements without throwing errors or wasting bullets, only engaging and auto-reloading for the valid targets within range.

Scenario 4: Overkill Damage

- **Concept:** An enemy has only 5 health remaining, but the tower lands a critical hit dealing 45 damage.
- **Expected Behavior:** The enemy's health will drop into the negative numbers. The system must recognize this "overkill" as a successful defeat and increment the defeated enemies counter.

Final Checklist for VPL

- [ ] **Inheritance Initialization:** Did you use super() correctly to initialize the base attributes in the SniperTower constructor?
- [ ] **Distance Mathematics:** Does your calculateDistance method correctly handle negative coordinates and return exactly 0.0 if the enemy is precisely on the tower?
- [ ] **Attribute Protection:** Does the fire method ensure that high enemy armor prevents negative damage *without* permanently modifying the tower's fundamental baseDamage attribute?
- [ ] **Independent Counters:** Does the critical hit mechanic accurately trigger on every 3rd shot, regardless of how many times the tower has reloaded?
- [ ] **Resource Conservation:** Does engageEnemies safely ignore out-of-range enemies without consuming any ammunition?
- [ ] **Infinite Loop Prevention:** Does engageEnemies immediately break the firing loop for a specific enemy if a shot deals exactly 0 damage?

[ ] **Overkill Recognition:** Does your code correctly count an enemy as defeated even if a high-damage critical hit drops their health well below 0?

Example Output

```
[Wave 1]
Enemies Defeated: 2 / 3
Remaining Ammo: 0

[Wave 2]
Enemies Defeated: 0 / 1
Remaining Ammo: 4

[Wave 3]
Enemies Defeated: 4 / 4
Remaining Ammo: 1
```

Requested files

towerDefense.java

```java
public class towerDefense {
    public static void main(String[] args) {

        // TEST 1
        System.out.println("[Wave 1]");
        double[][] wave1 = {
            {10.5, 20.0, 2.0, 50.0},
            {50.0, 50.0, 5.0, 100.0},
            {5.0, 5.0, 0.0, 30.0}
        };
        SniperTower tower1 = new SniperTower(0.0, 0.0, 30.0, 15, 5, 3);
        int defeated1 = tower1.engageEnemies(wave1);
        System.out.println("Enemies Defeated: " + defeated1 + " / 3");
        System.out.println("Remaining Ammo: " + tower1.getAmmo() + "\n");

        // TEST 2
        System.out.println("[Wave 2]");
        double[][] wave2 = {
            {10.0, 10.0, 50.0, 100.0}
        };
        SniperTower tower2 = new SniperTower(0.0, 0.0, 30.0, 15, 5, 3);
        int defeated2 = tower2.engageEnemies(wave2);
        System.out.println("Enemies Defeated: " + defeated2 + " / 1");
        System.out.println("Remaining Ammo: " + tower2.getAmmo() + "\n");

        // TEST 3
        System.out.println("[Wave 3]");
        double[][] wave3 = {
            {0.0, 5.0, 1.0, 10.0},
            {0.0, -5.0, 1.0, 10.0},
            {5.0, 0.0, 1.0, 10.0},
            {-5.0, 0.0, 1.0, 10.0}
        };
        SniperTower tower3 = new SniperTower(0.0, 0.0, 30.0, 15, 5, 3);
        int defeated3 = tower3.engageEnemies(wave3);
        System.out.println("Enemies Defeated: " + defeated3 + " / 4");
        System.out.println("Remaining Ammo: " + tower3.getAmmo());
    }
}
```

VPL